

Assignment 2: 路径规划算法实现

作业概述

在本次作业中，你需要修改 `path_planning_node.py` 中的 `plan_path` 函数，实现一个完整的全局路径规划算法（如 A* 或 Dijkstra），使得机器人能够自主避开障碍物到达目标点。

此外，你还需要使用提供的验证脚本 `verify_algo.py` 来评估你的算法性能。

任务列表

1. **代码理解**：阅读 `path_planning_node.py`，理解栅格地图与世界坐标的转换关系。
2. **算法实现**：
 - 在 `plan_path` 函数中删除现有的直线规划代码。
 - 实现 A* 搜索算法（或 Dijkstra）。
 - **要求**：路径不能穿过地图中的黑色障碍物区域。
3. **验证与测试**：
 - 运行 `verify_algo.py` 进行单元测试。
 - 在 Gazebo 仿真环境中运行完整系统进行实地测试。

验证脚本说明

我们提供了一个独立的 Python 脚本 `verify_algo.py`，它可以模拟 ROS 环境并直接测试你的算法逻辑。

如何运行

在终端中执行：

```
cd src/lab3_path_planning/scripts
python3 verify_algo.py
```

输出解读

脚本会运行两个测试用例：

1. **Direct Path**：起点和终点之间无障碍，测试算法的基础连通性。
2. **Cross Wall**：起点和终点之间有一堵墙，测试算法的避障能力。

如果你使用的是默认的“直线规划”代码，第二个测试用例会报告 **FAILURE - Collision detected**。当你成功实现 A* 后，它应该显示 **SUCCESS** 并给出路径长度。

提交要求

请提交以下文件：

1. 修改后的 `path_planning_node.py`。
2. 运行 `verify_algo.py` 的输出截图。

3. 在 RViz 中机器人成功绕过障碍物的截图。

评分标准

- **代码可运行** (20%)：没有语法错误，节点正常启动。
- **算法逻辑正确** (50%)：能够生成从起点到终点的连通路径。
- **避障功能** (30%)：生成的路径不穿过障碍物（通过验证脚本测试）。

提示

- 利用 `self.is_valid(grid_x, grid_y)` 函数来检查某个格子是否是障碍物。
- A* 算法的启发式函数可以使用欧几里得距离 (`self.heuristic`)。
- 注意处理边界情况（例如起点或终点在地图外，或者终点不可达）。